SOFTWARE TESTING ASSIGNMENT

Name: G. Ramani **Reg No.:** 192312279

Course code : CSA3718 **Course subject :** Software Testing

1. Identification and Documentation of Software Defects in an ATM System

1.1 Introduction

An Automated Teller Machine (ATM) allows customers to perform banking operations such as cash withdrawal, balance enquiry, fund transfer, PIN change and mini-statement printing. Since it handles money and confidential information, the ATM software must be accurate, secure, reliable and easy to use. This case study examines common potential defects based on typical ATM requirements. These are example defects to verify during testing, not claims that a particular bank's ATM currently contains them.

1.2 Basic System Requirements

- The ATM must authenticate a customer using a valid bank card and PIN.
- Cash withdrawal must be permitted only when the account has sufficient available balance and the transaction is within limits.
- The account must be debited only for a successfully completed withdrawal; failed or cancelled transactions must be handled safely.
- The ATM must display clear messages for invalid PINs, insufficient funds, unavailable services and technical failures.
- The system must protect PINs and account data and must end an inactive session securely.
- The ATM must keep transaction records and reconcile cash dispensed with the banking system.

1.3 Defect Identification and Documentation

Defect ID	Potential defect / condition	Expected behaviour	Severity	Suggested test
ATM-01	Account is debited even though no cash is dispensed.	Reverse or correct the transaction and inform the customer.	Critical	Simulate dispenser failure after authorization.
ATM-02	ATM dispenses more cash than the approved amount.	Dispense exactly the authorized amount, subject to available denominations.	Critical	Request a withdrawal and compare requested, recorded and dispensed cash.
ATM-03	Invalid PIN is accepted.	Reject invalid PINs and apply the configured retry/lock policy.	Critical	Enter an incorrect PIN.
ATM-04	PIN retry limit is not enforced.	Block further attempts according to bank policy.	High	Enter incorrect PINs repeatedly.
ATM-05	Withdrawal succeeds despite insufficient available balance.	Reject the transaction without debiting the account.	Critical	Attempt to withdraw more than the available balance.
ATM-06	Duplicate debit occurs after a timeout or retry.	A single customer request must not create duplicate debits.	Critical	Interrupt the network and retry the same request.
ATM-07	Balance is not refreshed after a successful withdrawal.	Display the updated balance or clearly state that it cannot be	High	Withdraw cash and immediately check balance.

		retrieved.		
ATM-08	Session remains active after card removal or timeout.	Terminate the session and require authentication again.	High	Leave the ATM idle or remove the card mid-session.
ATM-09	Sensitive account/PIN information appears on screen or receipt.	Mask or omit confidential information.	Critical	Inspect screens, receipts and logs during testing.
ATM-10	ATM accepts an amount that is not supported by available notes.	Explain the valid amount/denominations and ask the user to retry.	Medium	Request an unsupported amount.
ATM-11	Receipt shows incorrect amount, date or transaction status.	Receipt details must match the final transaction record.	High	Compare receipt with the transaction record.
ATM-12	ATM fails to return the card after cancellation or completion.	Return the card safely unless a documented security rule requires retention.	High	Cancel a transaction and test normal completion.

1.4 Defect Reporting Procedure

1. Record the defect ID, title, environment, date, build/version and tester.
2. Write clear preconditions, test data, reproduction steps and actual result.
3. Compare the actual result with the expected result and attach evidence where available.
4. Assign severity and priority, report the issue to the development team and track its status.
5. After the fix, retest the original scenario and run regression tests for related functions.

1.5 Conclusion

ATM testing should focus especially on financial correctness, authentication, transaction recovery, privacy and session handling. Documenting defects in a consistent format helps developers reproduce problems and helps testers verify that fixes do not introduce new failures.

2. ARMS Portal Case Study: Positive Test Scenarios and Test Cases

2.1 Introduction and Assumption

For this assignment, ARMS is assumed to mean an Academic Records Management System portal used by students and staff to sign in, view academic details, check attendance and results, and update permitted profile information. If the institution uses a different expansion or feature set for ARMS, the cases should be adjusted to its official requirements.

2.2 Requirement Analysis

Requirement ID	Requirement
FR-01	The portal shall allow registered users to log in using valid credentials.
FR-02	The portal shall show information appropriate to the user role, such as student or staff.
FR-03	Students shall be able to view their profile and enrolled course details.
FR-04	Students shall be able to view attendance records for available subjects/semesters.
FR-05	Students shall be able to view published examination results.
FR-06	Users shall be able to log out securely.
FR-07	The portal shall validate and save permitted profile changes.
NFR-01	Pages should load within an acceptable time under normal network conditions.
NFR-02	The portal shall protect personal and academic information and restrict unauthorized access.

2.3 Positive Test Scenarios

- Verify that a registered user can log in with the correct username and password.
- Verify that a student is directed to the student dashboard after successful login.
- Verify that the profile page displays the correct student details.
- Verify that enrolled subjects and semester information are displayed correctly.
- Verify that attendance details are displayed for a subject with available records.
- Verify that published examination results can be opened and viewed.
- Verify that permitted profile changes with valid data are saved successfully.
- Verify that the logout option ends the session and returns the user to the login page.
- Verify that navigation links open the correct pages.
- Verify that the portal works with supported browsers and a normal network connection.

2.4 Positive Test Cases

TC ID	Test case / precondition	Test steps and test data	Expected result
ARMS-P01	Login with valid credentials; registered student account exists.	Open portal; enter valid student ID and password; click Login.	Login succeeds and student dashboard appears.
ARMS-P02	Open student profile after login.	Select Profile from the menu.	Correct name, student ID, department and other authorized details appear.
ARMS-P03	View enrolled courses.	Open Courses/Subjects for the current semester.	Correct enrolled subjects and course details are listed.
ARMS-P04	View attendance with records available.	Open Attendance; choose a subject and semester.	Attendance percentage and session details match stored

			records.
ARMS-P05	View published results.	Open Results; select a semester with published results.	Correct subject-wise marks/grades and result status are displayed.
ARMS-P06	Save valid permitted profile change.	Edit an allowed field with valid data; click Save.	Success message appears and updated information persists after refresh.
ARMS-P07	Navigate between portal pages.	Open Dashboard, Profile, Attendance and Results using the menu.	Each link opens the correct page without an error.
ARMS-P08	Logout from an authenticated session.	Click Logout and confirm if prompted.	Session ends and login page is displayed.
ARMS-P09	Check dashboard summary.	Log in and inspect semester/course summary.	Summary values match the underlying academic records.
ARMS-P10	Open portal in a supported browser.	Use a supported browser and sign in with valid data.	Pages render correctly and core functions are usable.

2.5 Conclusion

Positive testing confirms that the ARMS portal performs its intended functions when valid users provide correct data and follow the expected workflow. The cases should be executed against the approved requirements, with actual results and evidence recorded for each test.

3. ARMS Portal Case Study: Negative Test Scenarios and Test Cases

3.1 Purpose

Negative testing checks how the ARMS portal behaves when users provide invalid input, lack permission, interrupt a workflow or encounter unexpected conditions. The system should reject invalid actions safely, display understandable messages and protect academic information.

3.2 Negative Test Scenarios

- Attempt login with an incorrect password or an unregistered user ID.
- Submit the login form with required fields left blank.
- Enter malformed data in a profile field, such as an invalid email address.
- Attempt to access another student's records by changing a URL or record identifier.
- Try to open a protected page after logging out or after the session expires.
- Attempt to edit a read-only field such as student ID or published marks.
- Attempt to view results that have not yet been published.
- Submit a profile form with values exceeding the allowed length or containing unsupported input.
- Refresh or resubmit a form repeatedly to check for duplicate or inconsistent updates.
- Simulate a network interruption while loading or saving information.
- Use browser navigation to return to a protected page after logout.
- Enter script-like input in text fields to verify that it is safely handled.

3.3 Negative Test Cases

TC ID	Invalid condition / test steps	Expected result
ARMS-N01	Enter a valid user ID with an incorrect password and click Login.	Login is rejected; a generic, clear error message appears; protected content is not shown.
ARMS-N02	Leave username and password blank; click Login.	Required-field validation appears and no login request is accepted.
ARMS-N03	Enter an unregistered student ID and a password.	Login fails without revealing whether sensitive account details exist.
ARMS-N04	Enter an invalid email format in an editable profile field and save.	Validation message appears; invalid data is not saved.
ARMS-N05	While logged in as Student A, change a record identifier in the URL to Student B's ID.	Access is denied; Student B's private academic information is not returned.
ARMS-N06	Log out, then use Back or revisit a protected URL.	Protected data is not accessible without a valid session.
ARMS-N07	Try to edit a read-only field such as student ID or published marks.	Change is rejected and the original record remains unchanged.
ARMS-N08	Open the results page for a semester whose results are not published.	The portal displays an appropriate "not published/available" message and does not show unauthorized data.
ARMS-N09	Enter an over-length value in a profile field and submit.	Input is validated safely; page remains stable and data is not corrupted.
ARMS-N10	Submit the Save button repeatedly or retry after a slow response.	The portal avoids duplicate updates and clearly indicates the final status.
ARMS-N11	Disconnect the network while saving a profile change.	A failure/retry message appears; the portal does not falsely report success or leave inconsistent data.
ARMS-N12	Enter HTML/JavaScript-like text into a text field and view the saved/displayed value.	Input is safely encoded or rejected; no script executes and no data is exposed.
ARMS-N13	Allow the session to expire, then attempt an action on a previously open page.	The user is redirected to login or shown a session-expired message; the action is not completed.

ARMS-N14	Enter only spaces in a required field.	The field is treated as empty and validation is shown.

3.4 Execution and Reporting

For each test case, record the test case ID, date, tester, environment, test data, actual result, status (Pass/Fail) and evidence. If the actual result differs from the expected result, raise a defect with reproduction steps and severity. Security tests must be performed only in an authorized test environment.

3.5 Conclusion

Negative testing helps ensure that the ARMS portal handles invalid data, unauthorized requests and unexpected failures safely. It is important for data confidentiality, data integrity and a reliable user experience.